Individual Assignment 2

Distributed BigEarthNet v2.0 Pipeline with Apache Spark

BDA501 – Big Data Analytics · Instructor: vubc (vubc@fpt.edu.vn) · Deadline: _____

1. Overview

Build an end-to-end pipeline with Apache Spark on **BigEarthNet-S2 v2.0**: 549,488 Sentinel-2 patches (120 × 120 px at 10 m) from 115 tiles over 10 European countries, each with multi-label land-cover annotations (19 classes). Records are small, but the data consists of **millions of tiny files**; handling this correctly is part of the task.

Step 1 Storage → **Step 2** Feature engineering → **Step 3** Spark SQL analysis → **Step 4** Classification with DataFrame/SQL

Use only **RDD, DataFrame and Spark SQL**. Spark MLlib is not required.

2. Data

- Download from `https://bigearth.net/#downloads`: `BigEarthNet-S2.tar.zst` (≈59 GiB) and `metadata.parquet` (recommended patches, i.e. without snow, cloud or shadow). Use the shared copy on the course HDFS if one is provided.
- Layout: `<TILE>/<PATCH_ID>/<PATCH_ID>_<BAND>.tif`. The tile name is `patch_id` without its last two `_`-separated tokens.
- `metadata.parquet` columns: `patch_id`, `labels` (array), `split` (train/validation/test), `country`, and quality flags.
- Use the 10 m bands B02 (Blue), B03 (Green), B04 (Red), B08 (NIR); values are `uint16` reflectance $\times 10^4$.

Pitfalls

(1) Each `.tif` holds *one* band: open one file per band. (2) Bands at 20 m/60 m have different sizes: do not stack them with the 10 m bands. (3) `uint16` subtraction wraps around: **cast to float before computing indices**.

Scale. Debug on a few hundred patches, then process **at least 50,000 patches from at least 3 countries** end-to-end (full dataset = bonus). Report the exact number of patches at every step.

3. Step 1 — Ingestion and Distributed Storage (20%)

- Store metadata and images in HDFS under `/bigearthnet/{raw, packed, metadata, features, results}/`.
- **Small-files problem:** pack the band files into Parquet (e.g. with `sc.binaryFiles` or `binaryFile`), one row per patch, partitioned by `country`.
- **Report:** number of patches (per country), total size and number of files *before and after* packing, block size / replication, and evidence of storage (`hdfs dfs -count`, `-du`, `-ls` or NameNode UI).

4. Step 2 — Distributed Feature Engineering (30%)

Produce one row per patch: `patch_id | country | split | labels | features...`, saved to `/bigearthnet/features/` (Parquet, partitioned by `country`).

Features (pixel values x_i as float reflectance, $N = 120 \times 120$, $\epsilon = 10^{-6}$):

- mean μ_b and standard deviation σ_b of each band B02, B03, B04, B08;
- $\overline{\text{NDVI}} = \frac{1}{N}\sum_i \frac{\text{B08}_i - \text{B04}_i}{\text{B08}_i + \text{B04}_i + \epsilon}$ and $\overline{\text{NDWI}} = \frac{1}{N}\sum_i \frac{\text{B03}_i - \text{B08}_i}{\text{B03}_i + \text{B08}_i + \epsilon}$ (per pixel, then averaged);
- brightness = mean of the four band means.

Spark requirement. Compute features on the executors with `rdd.mapPartitions` (or a UDF), decoding bytes in memory (`rasterio.io.MemoryFile`), then convert to a DataFrame with an explicit schema. Looping on the driver (`collect()`, `toPandas()`) is not accepted. A corrupt file must yield an error flag, not crash the job.

Validation and performance. Report: input vs. output row counts; nulls/NaN/∞ per column; `describe()` of all features (NDVI, NDWI must lie in $[-1, 1]$); a NumPy re-computation of 3 random patches matching the Spark output; run time for at least two partition counts, with a Spark UI screenshot.

5. Step 3 — Analysis with Spark SQL (20%)

Write at least three analyses as SQL queries on temporary views, and save all results to `/bigearthnet/results/`.

1. Class distribution (`explode(labels)`) and number of labels per patch.
2. Patches per country and per country × split.
3. Top-10 co-occurring class pairs.
4. Mean NDVI and brightness by country; mean of all features by class.
5. One question of your own.

Include at least three charts and comment on the results, especially class imbalance.

6. Step 4 — Classification with DataFrame / Spark SQL (30%)

1. **Task.** Pick 3–5 frequent classes. Keep patches containing *exactly one* of them; that class is the target.
2. **Split.** Use the official `split` column (no random split). All statistics used by the classifier come from **train** only; choose variants on validation; report on test.
3. **Rule-based baseline.** A `CASE WHEN` rule on at most three features, with thresholds derived from training data.
4. **Nearest-centroid classifier.** Standardise features with train mean/std; compute class centroids with `groupBy`; predict by `crossJoin` with the broadcast centroid table and pick the nearest centroid (window function).
5. **Evaluation (in Spark, not scikit-learn).** Confusion matrix (`groupBy().pivot()`), accuracy, per-class precision/recall/F1, macro-F1, compared with a majority-class baseline.
6. **Comparison.** Full feature set vs. a reduced set of 3–5 features (chosen on validation by correlation or class separation).
7. **Discussion.** Which classes are confused and why; which features matter; limits of the approach.

7. Deliverables

1. **Code**: notebooks/scripts per step, `README` with run commands, `requirements.txt`.
2. **Report** (PDF, 4–8 pages): environment (nodes, cores, memory), pipeline, storage, features, SQL results with charts, classification results, discussion.

Individual work. AI tools are allowed but must be cited; you must be able to explain your code orally.

8. Grading

Component	Weight	Key criteria
Step 1: Storage	20%	HDFS layout; small files packed and measured; statistics reported
Step 2: Features	30%	Computed on executors; correct and validated; performance evidence
Step 3: Spark SQL	20%	All analyses; results saved; charts with interpretation
Step 4: Classification	30%	No leakage; both classifiers; metrics vs. baseline; comparison; discussion
Bonus (max +10%)		Full dataset; Spark MLlib comparison; 20 m bands; measured Spark optimisation

Deductions: driver-side loop (−50% of Step 2); random split or statistics from validation/test (−30% of Step 4); results not reproducible (−20% overall); fewer than 50,000 patches (each step capped at 50%).